

Chapitre II : Classes et objets en JAVA

1. Programmation OO :

L'OO est paradigme de programmation permettant d'exprimer la solution en termes des éléments du problème à résoudre. Ceci présente un énorme pas en avant pour la résolution des problèmes complexes, et donc pour la **productivité** des programmeurs.

Les quatre principales caractéristiques de l'OO :

- Encapsulation : les méthodes d'accès
- Abstraction : L'abstraction est le processus qui consiste à représenter des objets qui appartiennent au monde réel dans le monde du programme que l'on écrit.
- Polymorphisme,
- Héritage,

2. Classes et objets en JAVA :

2.1 Définition :

- Dans un programme JAVA on manipule des objets.
- Un objet est caractérisé par :

	Objet			Vélo	
	Attributs/variables d'instance : partie statique déterminant l'état de l'objet			Vitesse Couleur Cadence de pédalage	
	Méthodes : partie dynamique déterminant le comportement de l'objet			Accélération Freinage Changement de vitesse	

- Le type de l'objet est appelé classe. Une classe est donc une structure de données qui :
 - Décrit les attributs et les méthodes des objets,
 - Permet aussi d'**encapsuler** les objets : voir opérateur de visibilité plus loin.

2.2 Déclaration d'une classe :

[Default][public][final][abstract] ClassNomClasse <i>// debut du corps de la classe</i>	Entête de la classe
<i>/* liste des attributs */</i> [public protected private][static][final] TypeAtt att1 ;	Les attributs
<i>/* le constructeur */</i> [public protected private] NomClasse(.....) { <i>/* corps du constructeur */</i> }	même nom que la classe sans type de retour
<i>/* liste des methods */</i> [public protected private][static][[abstract] TypeRetour meth1 (....) { <i>/*corps méthode */</i> } <i>}// fin de la classe</i>	Les méthodes

Exemple :

```
class Point {
int x;
int y;
Point(int dx, intdy){
x=dx;
y=dy;
}
void translate(intdx,intdy) {
x+=dx;
y+=dy;
}
```

3. Les objets:

3.1 Notion de référence :

En java, les variables sont soit d'un type de base, soit des références désignant des objets. Une référence est comparable à une adresse mémoire désignant un objet alloué dynamiquement.

3.2 Création des objets

En réalité, on distingue entre la déclaration et la création de l'objet.

- **Déclaration** : elle consiste à définir un identificateur et lui associer un type (classe). Cet identificateur servira comme une référence (adresse mémoire) d'un objet.

```
Point p1, p2 ; // déclaration de deux variables pour désigner deux objets de type Point
```

- **Création** : La création se fait à l'aide l'opérateur **new** et le constructeur.

```
p1= new Point(2,3) ; //création d'un objet  
p2 =p1 ; //p2 désigne le même objet que p1
```

3.3 Accès aux membres d'un objet :

L'accès aux méthodes d'une classe se fait par la notation "." sur une référence à un objet.

```
Class TestPoint {  
Public static void main ( String[] args ){  
Point p = new Point( );  
p.x= 2 ;  
p.y= 3 ;  
System.out.print("l'abscisse : " + p.x + " l'ordonné : p.y" );  
}
```

3.4 Le constructeur :

Le constructeur est une méthode spéciale appelé au moment de création d'un objet. Un constructeur :

- a le même nom que la classe,
- n'a pas de type de retour.

En général le rôle de constructeur est **d'initialiser les attributs et garantir les invariants**.

Si aucun constructeur n'est défini, le compilateur rajoute un constructeur **public** sans paramètres.

```
class Point {  
int x;  
int y;  
void translate(intdx,intdy) {  
x+=dx;  
y+=dy;  
}
```

```
Point p = new Point() ;  
p.x=2 ;  
p.y=3;
```

Appel inter-Constructeur:

- Il est possible d'avoir plusieurs constructeurs (on étudiera la surcharge des méthodes plus loin).
- Un constructeur peut faire appel à un autre appel avec le mot clé **this**.

Exemple :

<pre>class Point { int x; int y; Point(int dx, intdy){ x=dx; y=dy; } Point() { this(5,5) ; } voidtranslate(intdx,intdy) { x+=dx; y+=dy; }</pre>	<pre>public static void main (String[] arg) { Point p1 = new Point (); Point P2 = new Point (1,3); }</pre>
---	---

Le mot cléthis :

- Le mot-clé **this** désigne en permanence l'objet dans lequel on se trouve c.à.d. qu'il correspond à la référence de l'objet sur lequel la méthode courante a été appelée.
- Lorsqu'il n'y a pas d'ambiguïté, **this** est optionnel pour désigner un membre de l'instance courante.

<pre>class Point { int x; int y; Point(int x, int y){ this.x = x; this.y = y; } }</pre>

4. Les tableaux :

Les tableaux sont considérés **comme des objets**. Donc une variable de type tableau contient une référence d'un tableau. Cependant, JAVA a une syntaxe particulière pour la

- La déclaration des tableaux.
- Leur initialisation.

Déclaration : la taille n'est pas fixée	<code>int[] tabEntier ; //ou int[] tabEntiers[] ; // pas recommandé</code>
Création : on doit donner la taille	<code>tabEntiers = new int[3] ; // chaque élément reçoit par défaut zéro</code>
Initialisation :	<code>// declaration + création + initialisation int[] tabEntiers = {8, 2*3, 1} ; Point tabPoints = { new Point(2, 3), new Point(4, 4), } ;</code>
Affectation/utilization:	<code>tabEntiers[1] = 4 ; x=tabEntiers[1] ;</code>

Quelques fonction utiles :

Copier des éléments d'un tableau à un autre	<code>arraycopy</code>
Comparer 2 tableaux	
Une dimension	<code>equals</code>
2 dimensions ou plus	<code>deepEquals</code>
Trier	<code>sort</code>
Recherche par dichotomie	<code>binarySearch</code>
Remplir un tableau par une valeur	<code>fill</code>

5. Les chaînes de caractères :

On distingue 3 classes pour les chaînes de caractères :

- **String** pour les chaînes constantes,
- **StringBuilder** ou **StringBuffer** pour les chaînes variables.

5.1 La classe String :

*) Affectation :

```
chaîne = "Bonjour" ;

chaîne1 = "Bonjour" ; // un seul objet String référencé par deux variables

chaîne2 = new String("Bonjour") ; // new force la création d'une nouvelle chaîne
```

Remarque : String n'est pas modifiable, c.-à-d. une deuxième affectation crée une nouvelle chaîne.

***) Concaténation :**

- L'opérateur de concaténation : +
- Si l'un des opérandes est de type **String**, l'autre est traduit automatiquement en **String** :
 - o Les types primitifs sont traduits par le compilateur,
 - o Les objets sont traduits en utilisant la méthode **toString()**,

Remarque : s'il y a plusieurs + dans une expression, les calculs se font de gauche à droite.

```
int x = 5 ;
s = "Valeur de x + 1 = " + x + 1 ; //Valeur de x +1 = 51
s = x + 1 + "est la valeur de x + 1 " ; // 6 est la valeur de x + 1
```

***) Egalité de String :**

La méthode **equals** teste si 2 instances contiennent la même valeur.

```
String s1, s2 ;
s1 = "Bonjour" ; s2 = "les amis" ;
if ( (s1 +s2).equals("Bonjour les amis") ;
{ System.out.println("égales") ;
}
```

Remarque: "==" teste si les 2 objets ont la même adresse en mémoire, ne doit pas être utilisé pour comparer les chaînes.

Exemple de méthodes de String :

Récupérer le protocole, la machine, le répertoire et le nom du fichier d'une adresse Web.
`http://truc.unice.fr/rep1/rep2/fichier.html`

```
int n = adresse.indexOf(":"); // 4
String protocole = adresse.substring(0, n); // http
String reste = adresse.substring(n + 3);
n = reste.indexOf("/"); // 13
String machine = reste.substring(0, n);
String chemin = reste.substring(n + 1);
int m = chemin.lastIndexOf("/"); // 9
String repertoire = chemin.substring(0, m);
String fichier = chemin.substring(m + 1)
```

5.2 Chainesmodifibales :

StringBuffer ou **StringBuilder** possèdent des méthodes qui modifient le receveur du message et évitent la création de nouvelles instances ; par exemple :

- append et appendCodePoint
- insert
- replace
- delete

6. Surcharge des méthodes :

En JAVA, on peut surcharger une méthode, c.-à-d. ajouter une méthode qui a le même nom mais pas la même signature qu'une autre méthode.

```
class Point {  
...  
void translate(intdx,intdy) {  
x+=dx;  
y+=dy;  
void translate (Point p){  
x+=P.x ;  
y+=P.y ;  
}  
}
```

Remarque : Il est interdit de surcharger une méthode en changeant seulement le type de retour.

7. Variables/Méthodes de classe static:

7.1 Les variables de classe/static :

- Une variable de classe est une variable partagée par toutes les instances de la classe.
- Une variable de classe est accessible à partir d'un objet ou de la classe :

<pre>class Point { ... staticintnbrPoints =0 ; ... }</pre>	<pre>Point p = new Point() ; ... System.out.println(p.nbrpoints) ; //ou System.out.println(Point.nbrPoints) ;</pre>
--	---

- Elle est initialisée une seule fois lors de sa déclaration ou **dans un bloc static**.

<pre>class A { ... static int[] tab = new int[20] ; static { for(inti=0,i<25;i++) tab[i]=-1 ; } ... }</pre>	exécuté une seule fois quand la classe est chargée.
--	---

Question de réflexion :

Par analogie au bloc static, que fait le bloc **non-static** suivant :

```
class A {  
...  
int[] tab = new int[20] ;  
{  
for(int i=0, i<25; i++)  
tab[i]=-1 ;  
}  
.....  
}
```

7.2 Méthode de classe/static :

- Exécute une action indépendante d'une instance particulière.
- On exécute une méthode **static** en la préfixant par nom de la classe ou par une instance quelconque (à éviter car nuit à la visibilité).

Exemple :

<pre>static int getNbrPoints () { return nbrpoints ; }</pre>	<pre>Point p = new Point() ; Point.getNbrpoints() ; p.getNbrpoints() ;</pre>
---	--

Remarque :

- Une méthode de classe ne peut utiliser « **this** ».
- Une méthode classe ne peut avoir la même signature qu'une méthode d'instance.
- Une méthode de classe ne peut contenir des variables d'instances.

7.3 Compléments : Passage de paramètres :

- En JAVA le passage se fait en valeur : les valeurs des arguments sont recopiées dans l'espace mémoire de la méthode.
- Pour les objets, la valeur passée est une référence, donc :
 - Si la méthode modifie l'objet référencé, l'objet passé en argument sera modifié en dehors de la méthode.
 - Sinon, le changement de valeur d'un paramètre (primitif ou référence) n'a pas d'incidence en dehors de la méthode.

Exemple :

Le programme suivant affiche ??

<pre>public static void m(intip, Employe elp, Employe e2p) { ip = 100; elp.salaire = 80; e2p = new Employe("Emp1", 90); }</pre>	<pre>public static void main(String[] args) { Employe e1 = new Employe("Emp2", 100); Employe e2 = new Employe("Emp3", 120); inti = 10; m(i, e1, e2); System.out.println(i + '\n' + e1.salaire + '\n' + e2.nom); }</pre>
---	---

8. Les packages :

Les classes en JAVA sont regroupées en paquetages (bibliothèques). Ils offrent un niveau de modularité supplémentaire pour :

- Réunir des classes suivant leurs fonctionnalités,
- La protection des attributs et des méthodes.
- Selon la norme, le nom de paquetage ne prenne jamais de majuscule.

Importation :

Importation d'une classe	<code>import java.util.ArrayList ;</code>
Importation toutes les classes	<code>import java.util.* ;</code>
Importation variable/méthode static (depuis JDK 5)	<pre>import static java.lang.Math.* ; ... cos(PI)... import static java.lang.Math.PI ; Double x = PI * 2.3 ;</pre>

Ajout d'une classe dans un paquetage :

- Ajouter l'instruction `package nom-paquetage` au début du fichier source.

Exemple :

<pre>//Voiture.java package vehicule ; public class Voiture { ... } class Moto { ... } Emplacement : <...>/vehicule/Voiture.class <...>/vehicule/Moto.class</pre>	<pre>//Camion.java package vehicule ; public class Camion { ... } <...>/vehicule/Camion.class</pre>	<pre>// VoitureEssence.java Package vehicule.voiture ; public class VoitureEssence { ... } class VoitureDiesel{ ... } <...>/vehicule/voiture/VoitureEssence.class <...>/vehicule/voiture/VoitureDiesel.class</pre>
---	---	--

Complément sur la Compilation d'un paquetage :

Pour compiler : **javac -d rep_racine Classe.java**

- L'option «-d» permet d'indiquer le répertoire racine où sera rangé le fichier compilé : si on compile fichier qui comporte « **package nom1.nom2** » le fichier .class sera rangé dans le répertoire : **rep_racine/nom1/nom2**
- Sans l'option **-d** le fichier .class est rangé dans le même répertoire que le fichier source quel que soit le paquetage, (à éviter).

8. Encapsulation/ Visibilité :

- Permet d'assurer une indépendance entre l'implémentation des données et leurs utilisations.
- Cacher l'implémentation des données des objets.
- JAVA permet plusieurs degrés d'encapsulation :

private	seule la classe dans laquelle il est déclaré a accès.
public	Toutes les classes sans exception y ont accès.
Par défaut	Seules les classes de même paquetage (regroupement de classes)
protected	Sera étudié dans la section de l'héritage.

Protection de l'état interne d'un objet :

- Autant que possible l'état d'un objet doit être **private**.
- Pour la lecture d'une variable on lui associe un accesseur (getXXX()) avec un degré de visibilité que l'on veut.
- Pour la modification d'une variable on lui associe un modificateur (setXXX()) avec un degré de visibilité que l'on veut.
- Dans certains cas on peut autoriser l'accès au paquetage ou à tous :
 - Si la variable ne risque pas de recevoir des valeurs aberrantes,
 - Si on veut un accès rapide,
 - Pour simplifier l'écriture de code.

Exemple1 : coordonné polaire d'un point.

La classe point ne garantit pas l'encapsulation et on peut avoir des données aberrantes.

<pre> class Point { double rho; double theta; void init(double x,double y) { rho=Math.hypot(x,y); theta=Math.atan2(x,y); } } </pre>	<pre> class AutreClass { void autreMethod(Point p) { p.init(2.0,3.0); System.out.println(p.theta); p.rho=-1; // !!!!!!!! } } </pre>
---	--

Solution: utiliser modificateur

<pre> class Point { private double rho; private double theta; void init(double x,double y) { rho=Math.hypot(x,y); theta=Math.atan2(x,y); } void setRho(double r) { if(<0) /*générer exception */ else rho = r; } } </pre>	<pre> class AutreClass { void autreMethod(Point p) { p.init(2.0,3.0); System.out.println(p.theta); // ne compile pas p.rho=-1; // ne compile pas p.setRho(5.0) ; } } </pre>
---	--

Exemple2 : Implémentation d'un segment.

Cet exemple illustre la réutilisation des objets encapsulés.

Implémentation 1 :

<pre> Class Segment { private Point p1,p2 ; Segment(Point p1, Point p2){ this.p1=p1 ; this.p2=p2 ; } public setFstPoint(int x, int y){ p1.x= x ; p1.y=y ; } } </pre>	<pre> // autre classe utilise Segment Point p1=new Point(2,3) ; Point p2 = new Point(10, 5) ; Segment seg = new Segment(p1,p2) ; seg.setFstpoint(2.5) ; </pre>
---	--

Maintenant on suppose qu'on change l'implémentation du segment en utilisant un tableau de deux points. Dans ce cas on n'a pas besoin de changer toutes les classes qui utilisent Segment :

<pre>Class Segment { private Point[]tab ; Segment(Point p1, Point p2){ tab = new Point[2] ; tab[0] =p1 ; tab[1]= p2 ; } public setFstPoint(int x, int y){ tab[0].x= x ; tab[0].y=y ; } }</pre>	idem
--	------